Table of contents

SMT Solving

Introduction

SMT (Satisfiability Modulo Theories) extends the SAT problem to logical formulas including specific theories (arithmetic, arrays, etc.). For example, the formula:

$$n > 0 \implies n - 1 \leq n$$

Or, in a program verification context:

$$\neg(aux > 0) \land (res + aux = a + b) \land aux \geq 0$$

Tools like Why3 generate such formulas (via weakest precondition calculation, WP) and delegate their verification to an SMT solver.

For example, for a postcondition $P(a, b)$:

$$\underbrace{b \geq 0 \implies a + b = a + b \wedge b \geq 0}_{P(a,b)}$$

We want to prove: $\forall a, b, P(a, b)$.

We will explore: - The basic mechanisms of SMT solvers. - Their integration into tools like Why3.

SAT-Solving

The SAT Problem

Propositional logic manipulates boolean variables: $Var = \{p, q, ...\}$, which can be $true$ or $false$, and logical connectives: $\wedge, \vee, \implies, \neg$.

A PL formula is defined by the grammar: - $F \longmapsto P$ (variable) - $F \longmapsto \neg F$ - $F \longmapsto F \wedge F \mid F \vee F \mid F \implies F$

Note: Implication is right-associative: $p \implies q \implies t \iff p \implies (q \implies t)$.

Satisfying a PL Formula

An **interpretation** I is a function $I : Var \to \{true, false\}$.

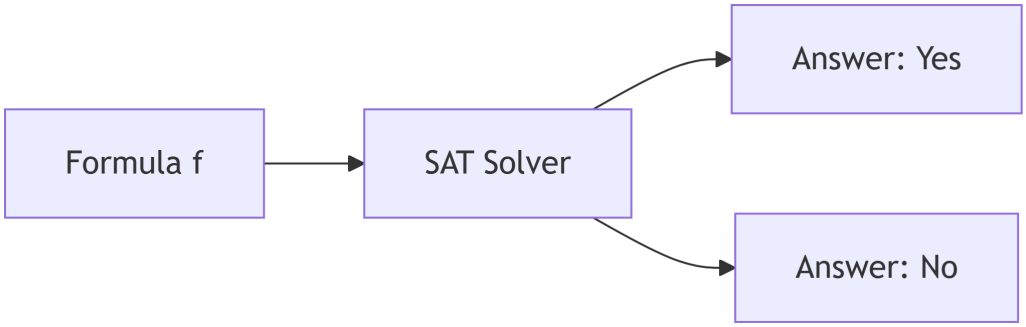
We extend I to any PL formula recursively: - If $f = p \in Var$, then $I(f) = I(p)$. - If $f = \neg f'$, then $I(f) = true$ iff $I(f') = false$. - If $f = f_1 \vee f_2$, then $I(f) = true$ iff $I(f_1) = true$ or $I(f_2) = true$. - If $f = f_1 \wedge f_2$, then $I(f) = true$ iff $I(f_1) = true$ and $I(f_2) = true$. - If $f = f_1 \implies f_2$, then $I(f) = true$ iff if $I(f_1) = true$ then $I(f_2) = true$ (otherwise $true$ if $I(f_1) = false$).

We say: - I **satisfies** f (denoted $I \vDash f$) iff $I(f) = true$. - f is **satisfiable** iff $\exists I, I \vDash f$. - f is **valid** (a tautology) iff $\forall I, I \vDash f$.

To prove that a formula g is valid, we can use **proof by refutation**: show that $\neg g$ is unsatisfiable. Often, g has the form $hypothesis \implies conclusion$, so $\neg g = hypothesis \wedge \neg conclusion$.

The SAT Problem

Given a PL formula f, the SAT problem is to decide if f is satisfiable.



- There are 2^n possible interpretations for n variables.
- The problem is **decidable** (algorithms exist) but **NP-complete**, so no efficient (polynomial) algorithm is known in general.
- In practice, solvers can fail due to resource constraints (time, memory).

An Old Method: Transformation to CNF

A **conjunctive normal form (CNF)** is a conjunction of clauses, where each clause is a disjunction of literals (a variable or its negation).

Definition: Equivalent CNF Formula

A formula is in CNF if it has the form:

$$C_1 \wedge C_2 \wedge \cdots \wedge C_n$$

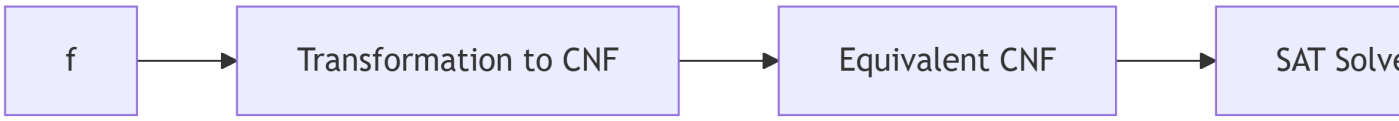
where each clause C_i has the form:

$$l_{i1} \vee l_{i2} \vee \cdots \vee l_{im}$$

and each literal l_{ij} is either a variable $p \in Var$, or its negation $\neg p$.

Example: $(p \vee \neg q) \wedge (r \vee s) \wedge \neg p$

We accept that for any PL formula f, there exists a PL formula in CNF that is logically equivalent. Therefore, we can restrict the SAT problem to CNF formulas.



Important remarks: - Simplification: a clause containing both a literal and its negation (e.g., $a \vee \neg a \vee ...$) is always true and can be removed. - **Unit clause**: a clause containing a single literal (e.g., a) forces the value of that variable. If $P \wedge a \wedge Q$ is true, then $a = true$ and we can simplify by resolving $P \wedge Q$.

DPLL Algorithm (1962)

Let P be a PL formula in CNF. For a literal x (variable or its negation), we denote $P[x]$ the formula obtained by imposing $x = true$ and simplifying.

The simplification rules are: - If x appears in a clause, that clause is removed (since satisfied). - If $\neg x$ appears in a clause, we remove $\neg x$ from that clause. - If a clause becomes empty, it is equivalent to $false$. - If a unit clause $\neg x$ remains, then x must be $false$, so $P[x] = false$.

DPLL(P) Algorithm:

```
If P = true, return true.
If P = false, return false.
If P contains a unit clause x, return DPLL(P[x]).
Choose a variable x in P.
Return DPLL(P[x]) or DPLL(P[¬x]).
```

Termination: The variant is the number of variables (decreases by 1 each call).

Correctness: Returns true iff P is satisfiable.

Example:

$$P = (p \vee q \vee r) \wedge (\neg q \vee r) \wedge \neg r \wedge (\neg p \vee q \vee r)$$

- DPLL(P):
  - $P[\neg r] = (p \vee q) \wedge \neg q \wedge (\neg p \vee q)$

  - $P[\neg r, \neg q] = p \wedge \neg p$
  - $P[\neg r, \neg q, p] = false$

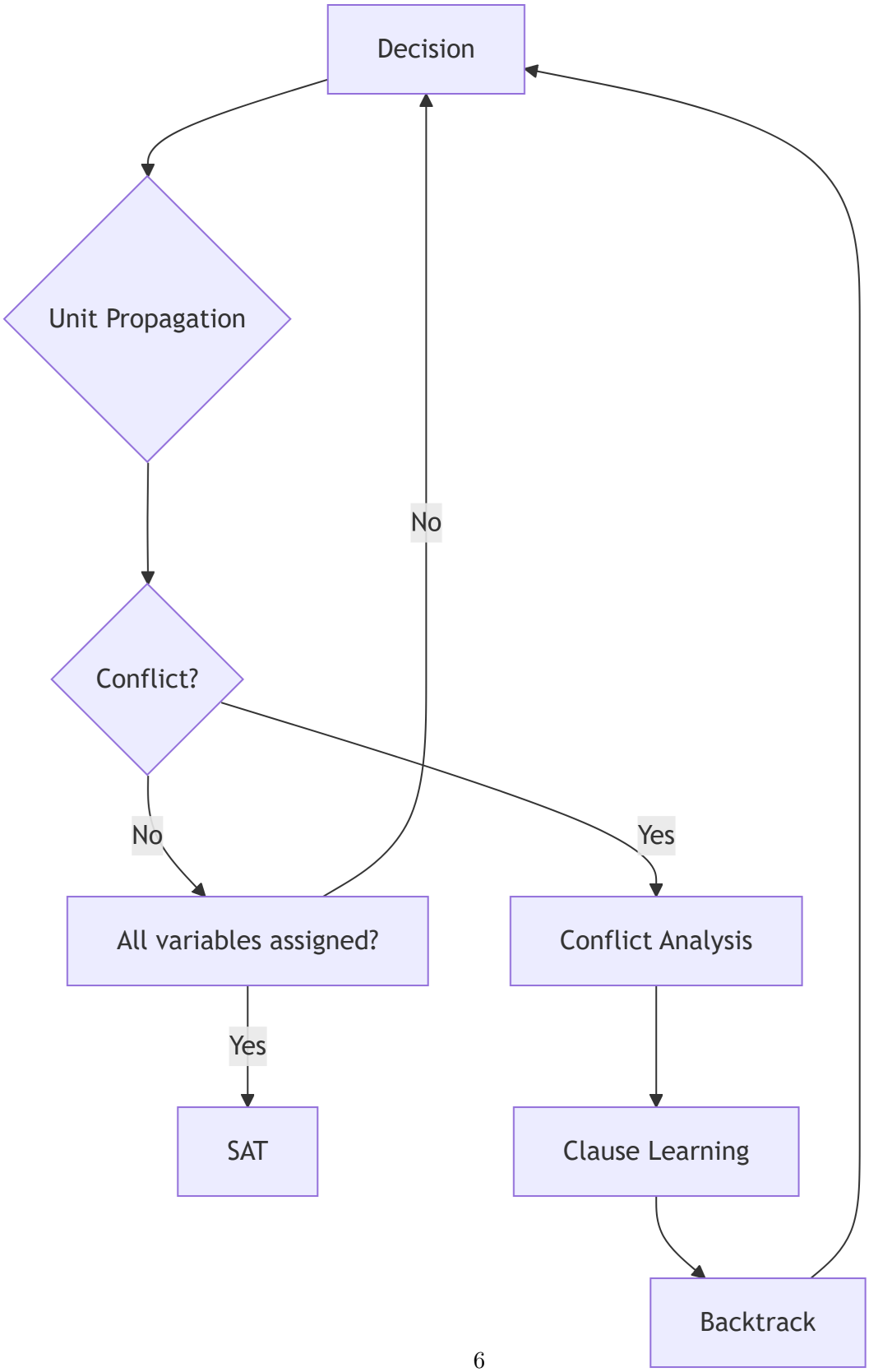
- Therefore P is **unsatisfiable**.

If DPLL returns true, we can reconstruct a model (an assignment of variables) by following the choices.

Modern Improvements: CDCL (1996-1999)

The **Conflict-Driven Clause Learning (CDCL)** algorithm extends DPLL with: - Learning clauses from conflicts. - Non-chronological backtracking. - A decision heuristic.

This allows handling problems with hundreds of thousands of variables and millions of clauses.


Decision
Unit Propagation
Conflict?
No
No
Yes
All variables assigned?
Conflict Analysis
Yes
SAT
Clause Learning
Backtrack

First-Order Logic and Theories

Let's return to the initial example:

$$a \geq 0 \land b \geq 0 \implies a + b \geq b + c$$

Here, the variables (a, b, c) are integers, and we use functions $(+)$ and predicates $(\geq)$. This is a **first-order logic (FOL) formula**.

FOL Syntax

- **Variables**: $Var = \{a, b, c, ...\}$.
- **Domain** D: a set of values (e.g., $\mathbb{Z}$).
- **Functions**: $\mathbf{F} = \{f : D^k \to D\}$ (includes constants, $k = 0$).
- **Predicates**: $\mathbf{P} = \{P : D^k \to \{true, false\}\}$.

Terms: - A variable is a term. - If $t_1, ..., t_k$ are terms and $f \in \mathbf{F}$ with arity k, then $f(t_1, ..., t_k)$ is a term.

Atoms: - $true$ and $false$ are atoms. - If $t_1, ..., t_k$ are terms and $p \in \mathbf{P}$ with arity k, then $p(t_1, ..., t_k)$ is an atom.

Formulas: - An atom is a formula. - If f_1, f_2 are formulas, then $\neg f_1$, $f_1 \land f_2$, $f_1 \lor f_2$, $f_1 \implies f_2$ are formulas. - If f is a formula and x a variable, then $\exists x f$ and $\forall x f$ are formulas.

Theories

A **theory T** is a set of FOL formulas that axiomatizes a particular domain (e.g., integers, arrays). A formula f is **T-valid** if it is a consequence of $\mathbf{T}$, i.e., for any interpretation I that satisfies $\mathbf{T}$ ($I \models \mathbf{T}$), we have $I \models f$.

Example: Integer Theory (Linear Arithmetic)

- Functions: $+$, $-$, constants.
- Predicates: $\geq$, $\leq$, $=$.
- Axioms: properties of addition, order, etc.

Example: Array Theory

- Domains: indices, values, arrays.
- Functions: $select(a, i)$ (read $a[i]$), $store(a, i, v)$ (write v at i).
- Axioms: $\forall a, i, v, select(store(a, i, v), i) = v$.

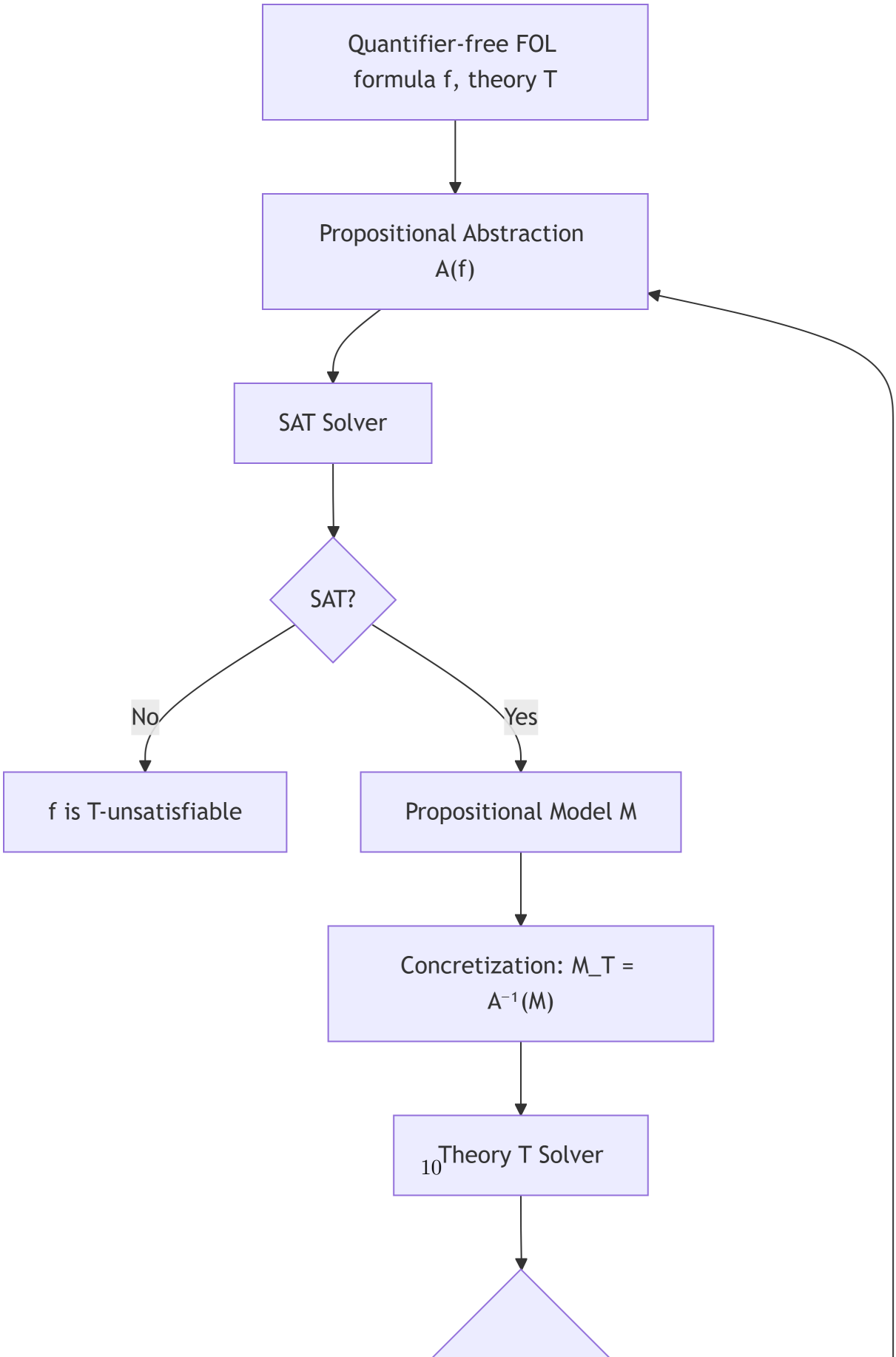
Example: Equality with Uninterpreted Functions (EUF)

- Predicate: $\approx$ (equality).
- Equivalence axioms: reflexivity, symmetry, transitivity.
- **Congruence**: for any function f and predicate p:
  - $\forall x, y, x \approx y \implies f(x) \approx f(y)$
  - $\forall x_1, y_1, x_2, y_2, x_1 \approx y_1 \wedge x_2 \approx y_2 \implies f(x_1, x_2) \approx f(y_1, y_2)$
  - $\forall x, y, x \approx y \implies (p(x) \iff p(y))$

SMT Solving: Combining SAT and Theories

The main idea: use a SAT solver for the propositional logical structure, and specialized solvers for the theories.

DPLL(T) Algorithm



Propositional Abstraction $A(f)$: we replace each atom of f with a new propositional variable.

Example:

$$f = (a + b \leq c \wedge a > 0 \Longrightarrow b + c < a) \vee a \leq 0$$

- $p_1 \leftrightarrow a + b \leq c$
- $p_2 \leftrightarrow a > 0$
- $p_3 \leftrightarrow b + c < a$
- $A(f) = (p_1 \wedge p_2 \Longrightarrow p_3) \vee \neg p_2$

If the SAT solver finds a model M (assignment of the p_i), we **concretize** it to $M_T = A^{-1}(M)$ (we replace the p_i with the corresponding atoms). We check if M_T is T-satisfiable using the theory solver.

- If yes, M_T is a model of f.
- If no, we add a **refutation clause** $\neg M$ (the negation of the assignment of the p_i) to $A(f)$ to avoid the same incorrect model, and we iterate.

Complete Example with EUF Theory

Consider the formula P:

$$g(a) = c \wedge (f(g(a)) \neq f(c) \vee g(a) = d) \wedge c \neq d$$

Propositional abstraction:

- $p_1 \leftrightarrow g(a) = c$
- $p_2 \leftrightarrow f(g(a)) = f(c)$
- $p_3 \leftrightarrow g(a) = d$
- $p_4 \leftrightarrow c = d$
- $A(P) = p_1 \wedge (\neg p_2 \vee p_3) \wedge \neg p_4$

The SAT solver may find the model $M = p_1 \wedge \neg p_2 \wedge \neg p_4$.

Concretization: $M_T = g(a) = c \wedge f(g(a)) \neq f(c) \wedge c \neq d$.

The EUF solver detects that $g(a) = c$ implies $f(g(a)) = f(c)$ (congruence). Therefore M_T is EUF-unsatisfiable.

We add the refutation clause: $\neg M = \neg p_1 \vee p_2 \vee p_4$.

The new SAT formula is: $p_1 \wedge (\neg p_2 \vee p_3) \wedge \neg p_4 \wedge (\neg p_1 \vee p_2 \vee p_4)$.

The SAT solver can now find another model, until a T-satisfiable solution is found or the SAT formula becomes unsatisfiable (which proves that P is T-unsatisfiable).

FOL Formulas with Quantifiers

Definitions

- **Universal formula**: of the form $\forall x_1, ..., x_k \phi$, where ϕ is quantifier-free.
- **Closed formula**: every variable is bound by a quantifier.
- **Closed instance**: for a quantifier-free formula ϕ, a closed instance is obtained by a substitution θ that replaces each variable with a closed term (without free variables). We denote $\theta(\phi)$.

Herbrand's Theorem (1930)

A set of quantifier-free FOL formulas is unsatisfiable **iff** there exists a finite set of closed instances that is unsatisfiable.

This reduces the unsatisfiability of a closed universal formula to the unsatisfiability of a finite set of propositional formulas (by instantiating variables with terms from the domain).

Semi-decidable algorithm (TestSat):

```
F := { }
while F is satisfiable do
   choose a substitution   such that  ( ) is a closed instance
   F := F   { ( )}
end while
return "  is unsatisfiable"
```

If ϕ is satisfiable, the loop may never terminate. **Guided instantiation** techniques (trigger, E-matching) are used in practice.

Example

Consider the syllogism: - All humans are mortal. - All Greeks are human. - Therefore, all Greeks are mortal.

Formally, we want to prove that the following formula is valid:

$$(\forall x, H(x) \implies M(x)) \wedge (\forall x, G(x) \implies H(x)) \implies (\forall x, G(x) \implies M(x))$$

By refutation, we consider its negation:

$$(\forall x, H(x) \implies M(x)) \land (\forall x, G(x) \implies H(x)) \land (\exists x, G(x) \land \neg M(x))$$

We transform it into a closed universal formula (step by step):

1. **Skolemization**: replace $\exists x$ with a fresh constant s.

$$(\forall x, H(x) \implies M(x)) \land (\forall x, G(x) \implies H(x)) \land (G(s) \land \neg M(s))$$

2. **Put in prenex form** (all quantifiers at the beginning):

$$\forall x, y, (H(x) \implies M(x)) \land (G(y) \implies H(y)) \land G(s) \land \neg M(s)$$

3. **Form without internal quantifiers** (we can remove universal quantifiers for now, since we will instantiate):

$$\phi = (H(x) \implies M(x)) \land (G(y) \implies H(y)) \land G(s) \land \neg M(s)$$

Now, we apply Herbrand. The Herbrand universe consists of the constant s. We take the closed instance by replacing x and y with s:

$$\theta(\phi) = (H(s) \implies M(s)) \land (G(s) \implies H(s)) \land G(s) \land \neg M(s)$$

Is this propositional formula satisfiable? - From $G(s)$ and $G(s) \implies H(s)$, we deduce $H(s)$. - From $H(s)$ and $H(s) \implies M(s)$, we deduce $M(s)$. - But we have $\neg M(s)$. Contradiction.

Therefore $\theta(\phi)$ is unsatisfiable, which proves that the original formula is valid.

Transformation to Closed Universal Formula

To apply Herbrand, we often need to transform an arbitrary FOL formula into a closed universal formula that preserves unsatisfiability.

Procedure: 1. Eliminate $\implies$ and $\iff$ using $\neg, \land, \lor$. 2. Rename variables so that each quantifier binds a fresh variable. 3. **Skolemization**: replace each existential variable $\exists y$ with a term $f(x_1, ..., x_k)$ where f is a new function symbol (a constant if $k = 0$) and $x_1, ..., x_k$ are the universal variables enclosing $\exists y$. 4. Move all universal quantifiers to the front (prenex form).

Example of skolemization: - $\forall x, \exists y, P(x, y)$ becomes $\forall x, P(x, f(x))$ (with f fresh). - $\exists y, Q(y)$ becomes $Q(c)$ (with c fresh). - $\forall x, y, \exists z, R(x, y, z)$ becomes $\forall x, y, R(x, y, g(x, y))$ (with g fresh).

This transformation preserves unsatisfiability, which is sufficient for proof by refutation.

Conclusion

SMT solvers combine the power of modern SAT solvers (CDCL) with decision procedures for various theories (EUF, arithmetic, arrays, etc.).

For formulas with quantifiers, instantiation techniques based on Herbrand's theorem are used, often guided by triggers and E-matching.

These methods are at the core of deductive verification tools like Why3, which generate proof obligations discharged to SMT solvers.